

Protocolos estándar y servicios REST

1. Protocolos estándar y servicios REST: leyendo el controlador

Objetivo de la sesión

Ya sabes...

Dos programas pueden comunicarse usando una IP y un puerto. Para entenderse necesitan reglas compartidas.

Hoy añadimos...

HTTP, URL, verbos, códigos de estado, JSON y la traducción de todo ello a un controlador REST en Spring.

No vas a programar todavía: vas a aprender a leer un controlador REST ya construido.

Pregunta guía: ¿cómo acaba GET /api/v1/libros/3 ejecutando un método Java y devolviendo JSON?

La URL, pieza a pieza

Una URL indica dónde está el servidor y qué recurso concreto queremos dentro de él.

http://

Protocolo

Reglas de
comunicación

localhost:8080

Host + puerto

Servidor y puerta de entrada

/api/v1/libros/3

Ruta

Recurso solicitado

En una API REST, la ruta identifica qué recurso queremos y el verbo HTTP la operación queremos realizar.

Anatomía de una conversación HTTP

El cliente envía una petición y el servidor responde con un recurso.



Una petición y una respuesta HTTP reales

Petición del cliente

```
GET /api/v1/libros/3 HTTP/1.1
Host: localhost:8080
Accept: application/json
```

Respuesta del servidor

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id":3,
  "titulo":"El nombre del viento",
  "precio":19.95
}
```

Primera línea

método + ruta / código de estado

Cabeceras

metadatos sobre el mensaje

Cuerpo

datos enviados, puede no existir

Accept = formato que el cliente quiere recibir · Content-Type = formato real del cuerpo que se está enviando.

Verbos HTTP e idempotencia

El verbo expresa la intención. La idempotencia describe qué ocurre si repetimos la misma operación.

GET

Leer

Idempotente: Sí

POST

Crear

Idempotente: No

PUT

Reemplazar

Idempotente: Sí

PATCH

Modificar
parcialmente

Idempotente:
Depende

DELETE

Eliminar

Idempotente: Sí

¿Qué significa idempotente?

Repetir la petición produce el mismo efecto final sobre los datos que ejecutarla una sola vez.

Ejemplo con PATCH

“Pon el precio en 19,95 €” puede ser idempotente. “Aumenta el precio en 5 €” no lo es.

Una novedad del estándar: el método QUERY

¿Para qué sirve?

Pensado para consultas cuyos criterios son demasiado complejos o extensos para expresarlos cómodamente en la URL.

El cuerpo puede describir la consulta, por ejemplo en JSON.

¿Lo usaremos aquí?

No. En este curso las consultas se implementarán con GET y parámetros en la URL, porque Spring todavía no ofrece soporte directo mediante `@QueryMapping` ni `RequestMethod.QUERY`.

Conocer que existe no significa tener que usarlo: nos interesa identificar el estándar y, a la vez, trabajar con herramientas soportadas por el framework.

Códigos de estado: la respuesta en tres cifras

2xx

Éxito

200 OK
201 Created
204 No Content

4xx

Problema en la petición

400 Bad Request
401 / 403
404 Not Found

5xx

Problema en el servidor

500 Internal Server
Error

No hace falta memorizar todos los códigos: aprende a leer la familia y reconoce los habituales.

JSON como formato del cuerpo

En las APIs modernas, JSON es el formato habitual para representar datos dentro del cuerpo HTTP.

```
{  
  "titulo": "El nombre del viento",  
  "precio": 19.95,  
  "editorial": {  
    "nombre": "Plaza & Janés"  
  }  
}
```

Pares clave-valor

Cada dato tiene un nombre y un valor.

Puede anidarse

Un valor puede ser otro objeto, como editorial.

Aquí solo lo reconocerás

Lo usarás de forma práctica al construir peticiones en la actividad.

API + REST: recursos, rutas y verbos

API

Conjunto de operaciones que una aplicación expone para que otros programas puedan utilizarla sin conocer su implementación interna.

REST

Estilo para diseñar APIs sobre HTTP: modela los datos como recursos identificados por URL y usa verbos HTTP para operar sobre ellos.

GET

/libros

listar

GET

/libros/3

leer uno

POST

/libros

crear

PUT

/libros/3

reemplazar

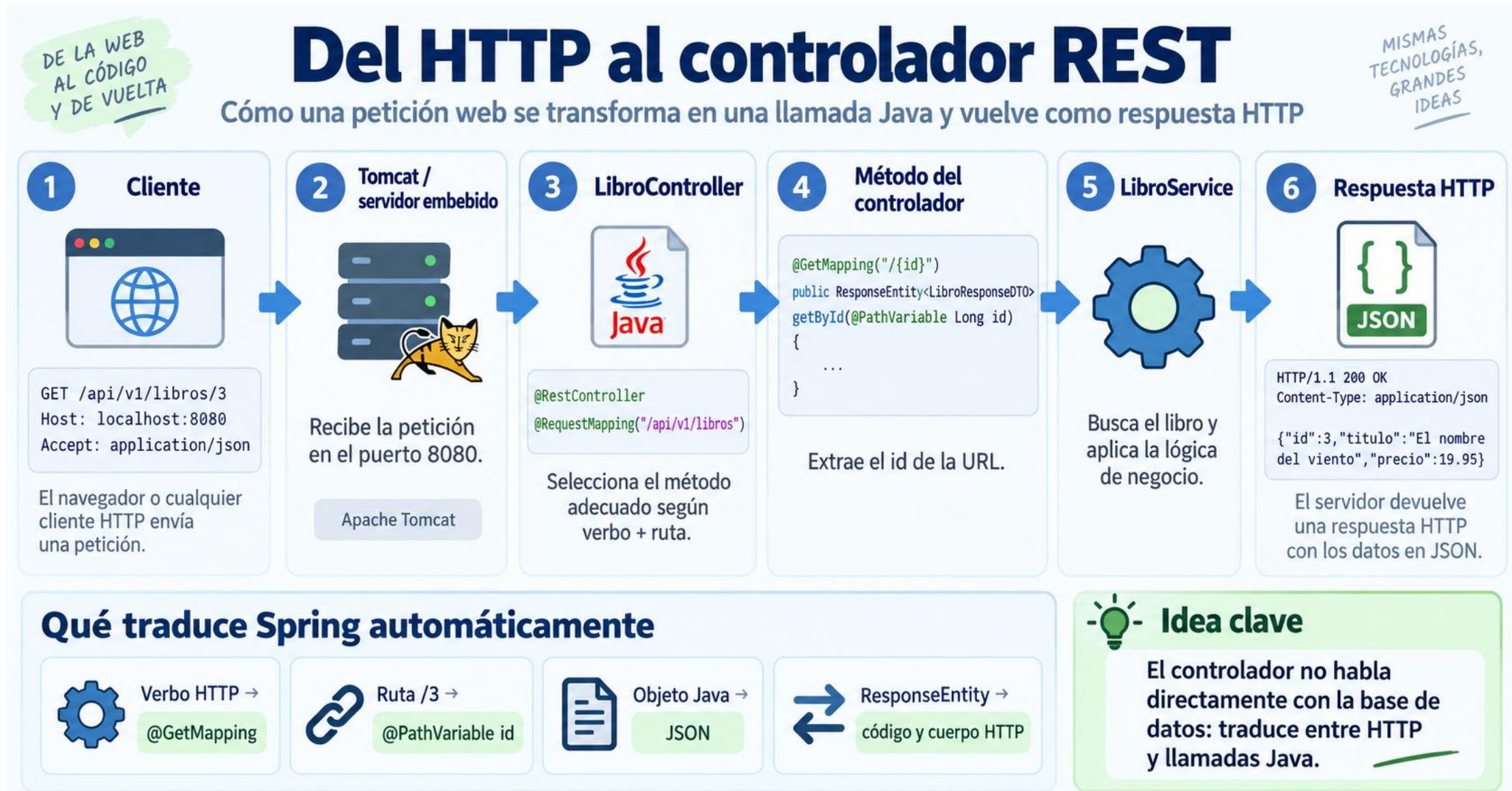
DELETE

/libros/3

borrar

La ruta identifica qué recurso; el verbo identifica qué operación. Evita rutas tipo /libros/borrar/3.

Del HTTP al controlador REST



El controlador completo

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService libroService;

    @GetMapping
    public ResponseEntity<List<LibroResponseDTO>> getAll() {
        return ResponseEntity.ok(libroService.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<LibroResponseDTO>
        getById(@PathVariable Long id) {
        return ResponseEntity.ok(libroService.findById(id));
    }
}
```

Qué hace el controller

Traduce entre HTTP y llamadas Java.

Qué no hace

No consulta directamente la base de datos.

Quién trabaja

LibroService aplica la lógica y obtiene el resultado.

Anotación a anotación: qué traduce cada una

```
@RestController
```

Clase que recibe peticiones y serializa la respuesta al cuerpo HTTP.

```
@RequestMapping("/api/v1/libros")
```

Ruta base común a los métodos del controlador.

```
@GetMapping("/{id}")
```

Verbo GET + parte variable de la ruta.

```
@PathVariable Long id
```

Extrae el 3 de /libros/3 y lo entrega como Long.

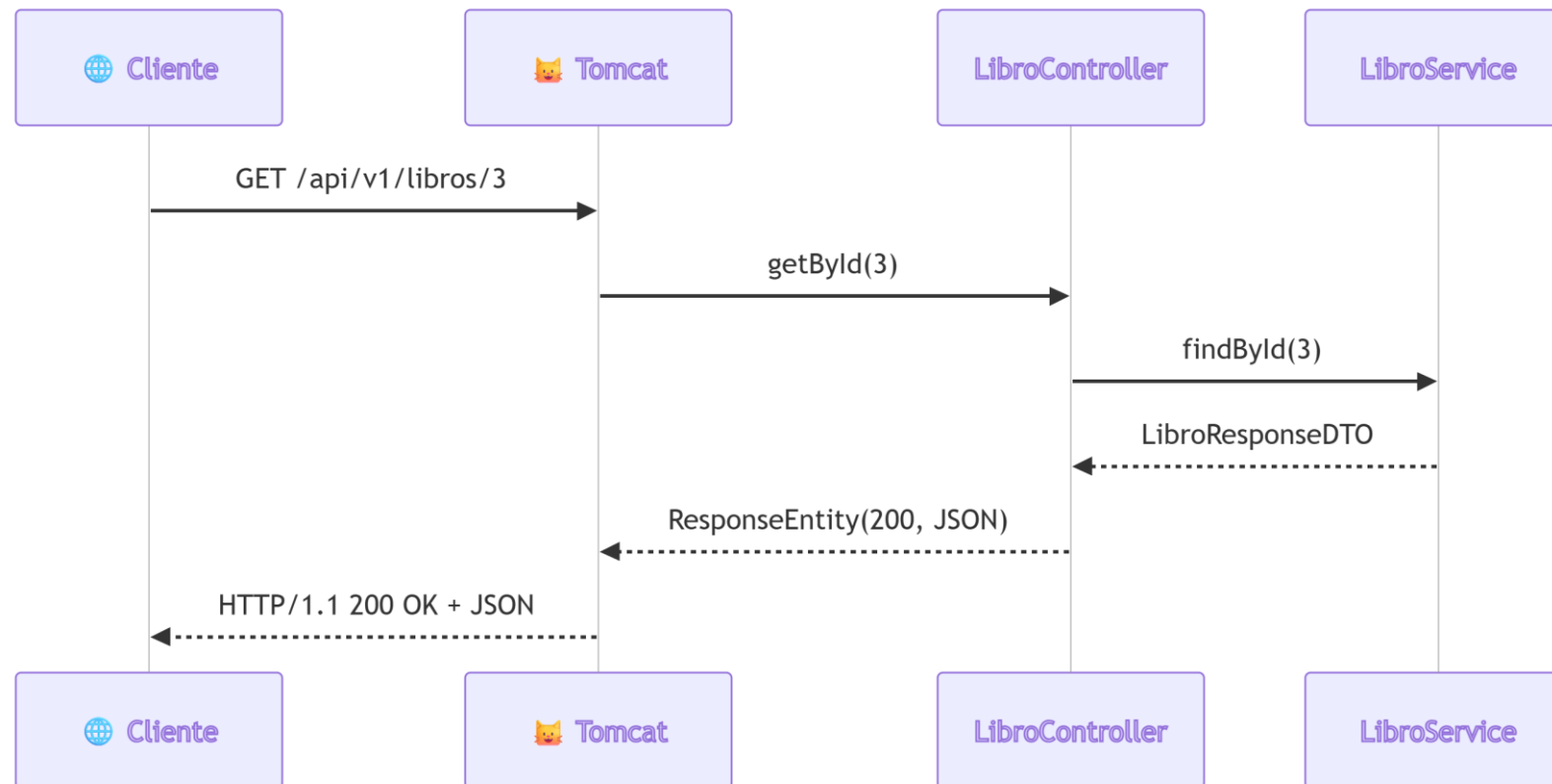
```
ResponseEntity.ok(...)
```

Construye una respuesta 200 OK con un cuerpo.

No hay magia añadida: GET + ruta + dato variable se convierten en la selección de un método y sus parámetros.

El viaje completo, ida y vuelta

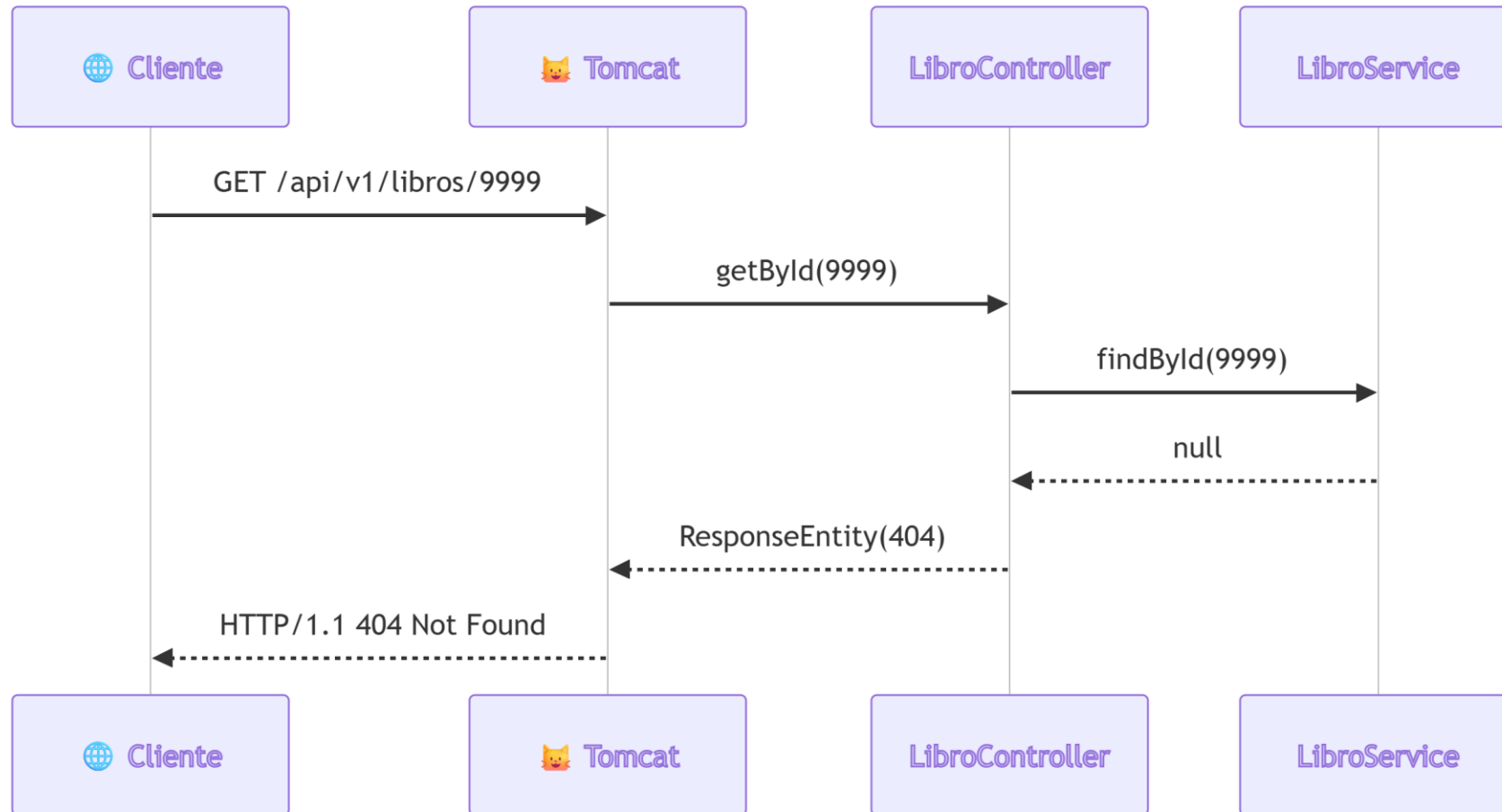
Fíjate en que la respuesta recorre exactamente el mismo camino que la petición, pero al revés:



El puerto 8080 y Tomcat no son magia

El servidor Tomcat que escucha en el puerto 8080 lo trae la dependencia `spring-boot-starter-webmvc` del `pom.xml` — es la "librería que implementa el servicio en red" de la que habla el currículum. Tú no arrancas ningún servidor a mano: Spring Boot lo hace por ti al ejecutar la clase anotada con `@SpringBootApplication`.

¿Y si el recurso no existe?



Éxito vs 404: misma petición, distinto resultado

✓ Recurso encontrado

```
GET /api/v1/libros/3
```

LibroService encuentra el libro.
El controller construye una respuesta correcta.

```
HTTP/1.1 200 OK  
Content-Type: application/json  
{ "id": 3, ... }
```

✗ Recurso no encontrado

```
GET /api/v1/libros/9999
```

LibroService no encuentra el libro.
El controller traduce ese resultado a HTTP.

```
HTTP/1.1 404 Not Found  
  
(sin cuerpo en esta primera versión)
```

404 no significa que el servidor esté caído: significa que ese recurso no se ha encontrado.

Cómo el código produce el 404

```
@GetMapping("/{id}")
public ResponseEntity<LibroResponseDTO>
    getById(@PathVariable Long id) {

    LibroResponseDTO libro = libroService.findById(id);

    if (libro == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(libro);
}
```

Si existe

`ResponseEntity.ok(libro)` → 200 OK + cuerpo

Si no existe

`ResponseEntity.notFound().build()` → 404 sin cuerpo

Más adelante el service lanzará una excepción con `orElseThrow()` y evitaremos repetir este if.

Por qué un protocolo estándar

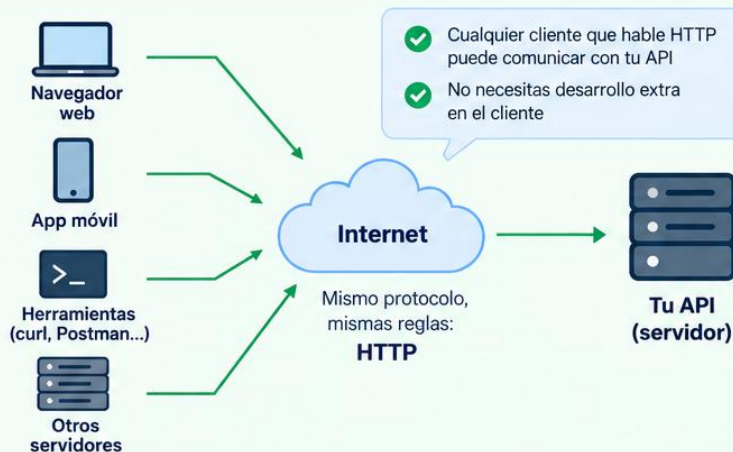
Por qué usar un protocolo estándar (HTTP) en tus APIs

Comparativa con un protocolo propio



HTTP / REST (protocolo estándar)

Un lenguaje común que ya conoce todo el mundo



Clientes ya existentes

Navegadores, apps, herramientas y librerías en casi todos los lenguajes.



Herramientas y documentación estándar

curl, Postman, Swagger/OpenAPI, monitores, proxies, etc.



Más fácil de integrar

Otros sistemas y empresas ya saben cómo comunicarse por HTTP.



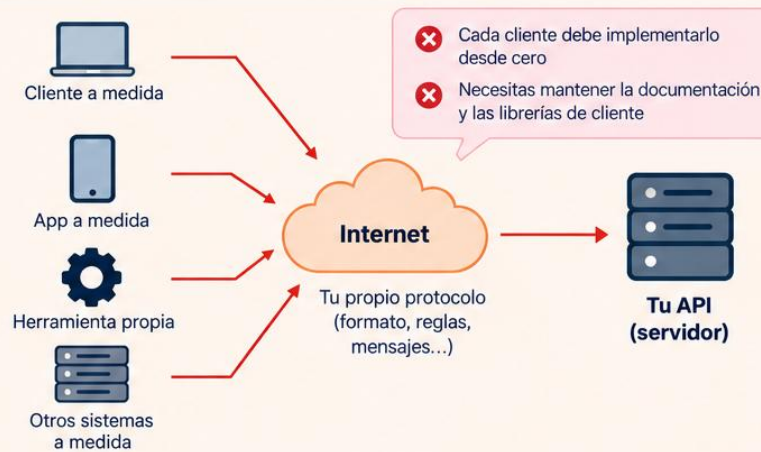
Menor coste de mantenimiento

Sigues un estándar ampliamente probado y evolucionado.



Protocolo propio

Tendrás que definir tu propio lenguaje de comunicación



Clientes a medida

Cada cliente (web, móvil, otro servidor...) debe implementar tu protocolo.



Sin herramientas estándar

No puedes usar directamente curl, Postman, navegadores, etc.



Mayor coste de integración

Otros equipos o empresas deben aprender tu protocolo y desarrollar clientes.



Más mantenimiento

Debes documentar, versionar y mantener el protocolo y las librerías de cliente.



Idea clave

Usar un protocolo estándar como HTTP te permite centrarte en la lógica de tu aplicación, reutilizar herramientas existentes y facilitar que otros sistemas puedan integrarse con tu API.

"Los estándares no se inventan para complicar, sino para que todo encaje más fácil."

Ideas clave

- Toda comunicación HTTP es petición-respuesta entre un cliente y un servidor.
- La URL localiza el recurso; el verbo expresa la operación.
- HTTP organiza cada mensaje en primera línea, cabeceras y cuerpo opcional.
- Los códigos 2xx, 4xx y 5xx resumen el resultado.
- REST modela datos como recursos y utiliza las convenciones de HTTP.
- Spring traduce verbo + ruta en un método Java y ResponseEntity construye la respuesta.
- Usar un estándar permite reutilizar clientes, herramientas y conocimiento existente.

Si puedes seguir el recorrido GET /api/v1/libros/3 → getById(3) → 200/404, tienes el núcleo de la sesión.